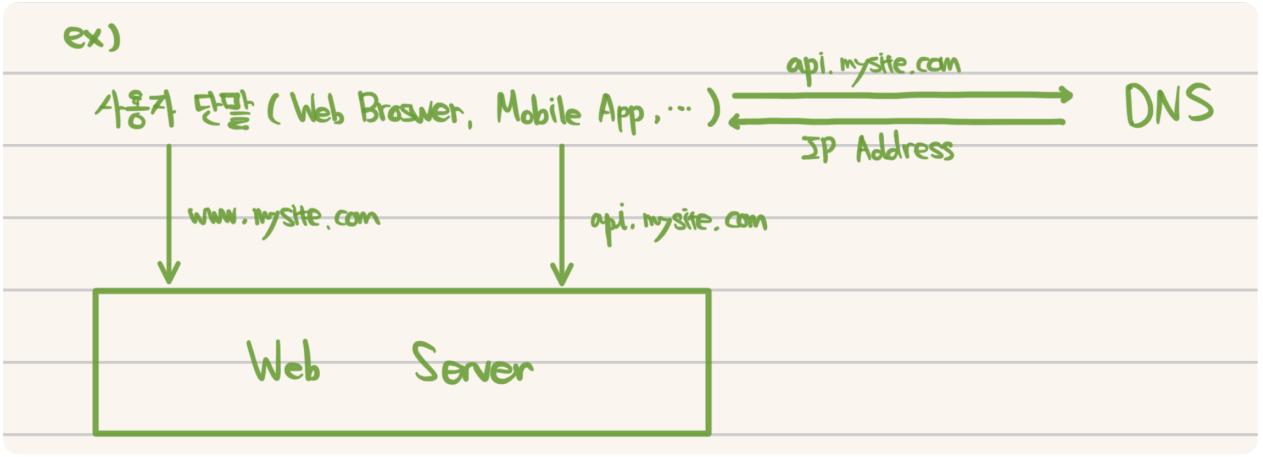
1. 사용자 수에 따른 규모 확장성

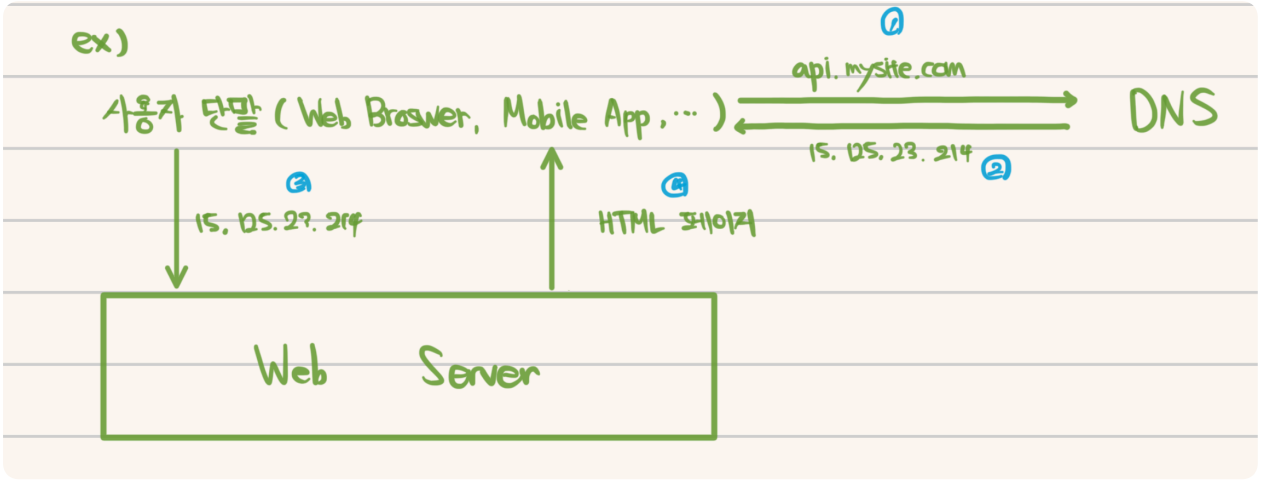
1.1 단일 서버

> 웹앱, DB, 캐시 등이 전부 서버 한 대에서 실행된다.

예제



이 시스템 구성을 이해하기 위해서는 **사용자의 요청이 처리되는 과정과 요청을 만드는 단말**에 대해서 이해해야한다.



(1)

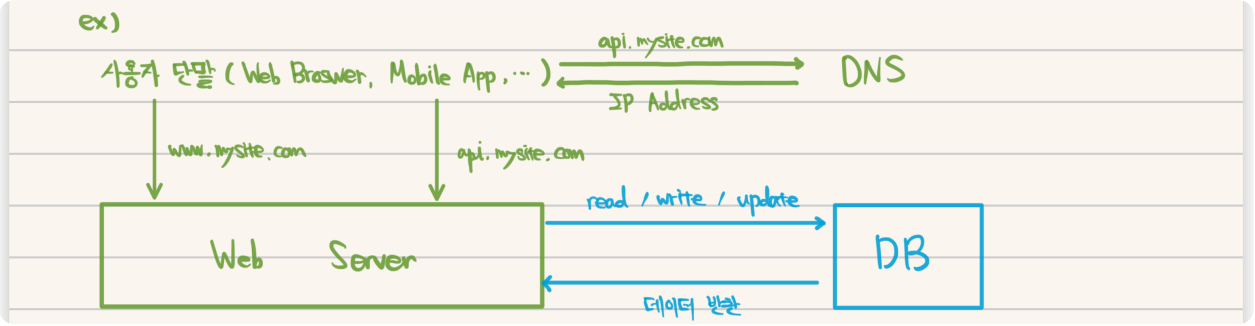
사용자는 도메인 이름(api.mysite.com)을 이용해 웹 사이트에 접속한다. 이 접속을 위해 도메인 이름을 Domain Name Service (DNS) 에 질의하여 IP주소로 변환한다. DNS 는 보통 제 3 사업자가 제공하는 유료 서비스를 이용한다.

(2) DNS 조회 결과로 IP 주소가 변환된다.
(3) 해당 IP 주소로 HTTP(HyperText Transfer Protocol) 요청이 전달된다.
(4) 요청 받은 웹 서버는 HTML 페이지나 JSON 형태의 응답을 반환한다.

> DNS 에 관한 부가 설명 :

1.2 데이터베이스

사용자가 증가하면 서버 하나로는 충분하지 않아서 여러 서버를 두어야한다.
하나는 웹·모바일 트래픽 처리 용도고, 다른 하나는 DB 용도이다.
이 둘을 분리하면 각각을 독립적으로 확장해 나갈 수 있게 된다.



> 어떤 데이터베이스를 사용할까? 라고 질문할 수 있다.
> 우리는 관계형 · 비 관계형 데이터베이스 사이에서 고를 수 있다.

1.2.1 관계형

가장 유명한 것으로는 MySQL, Oracle, PostgreSQL 등이 있다.
관계형에서는 자료를 테이블과 열·컬럼으로 표현한다. SQL 을 사용하면 여러 테이블에 있는 관계에 따라 조인하여 합칠 수 있다.

1.2.2 비 관계형

비 관계형 데이터베이스는 NoSQL 이라고도 부른다.
대표적으로 CouchDB, Neo4j, Cassandra, HBase 등이 있다.
NoSQL 은 다시 네 부류로 나눌 수 있다.

- Key Value
- Graph
- Column
- Document

> 이런 비 관계형 데이터베이스는 일반적으로 조인 연산은 지원하지 않는다.

하지만 관계형 데이터베이스가 적합하지 않은 경우에는 관계형 데이터베이스 이외의 저장소도 살펴보아야한다.

비 관계형 데이터베이스가 괜찮은 경우

- 아주 낮은 응답 지연시간이 요구된다
- 다루는 데이터가 비정형이라 관계형 데이터가 필요 없는 경우
- 데이터를 직렬화와 역직렬화 할 수 있기만 한 경우
- 아주 많은 양의 데이터를 저장할 필요가 있을 때

1.3 수직적 규모 확장과 수평적 규모 확장

> **Scale up** : 수직적 규모 확장으로 서버에 고사양 자원을 추가하는 행위이며 여기서 고사양 자원이라고 하는 것은 더 좋은 CPU, 더 많은 RAM 을 의미한다.

> **Scale out** : 수평적 규모 확장으로 더 많은 서버를 추가하여 성능을 개선한느 행위이다.

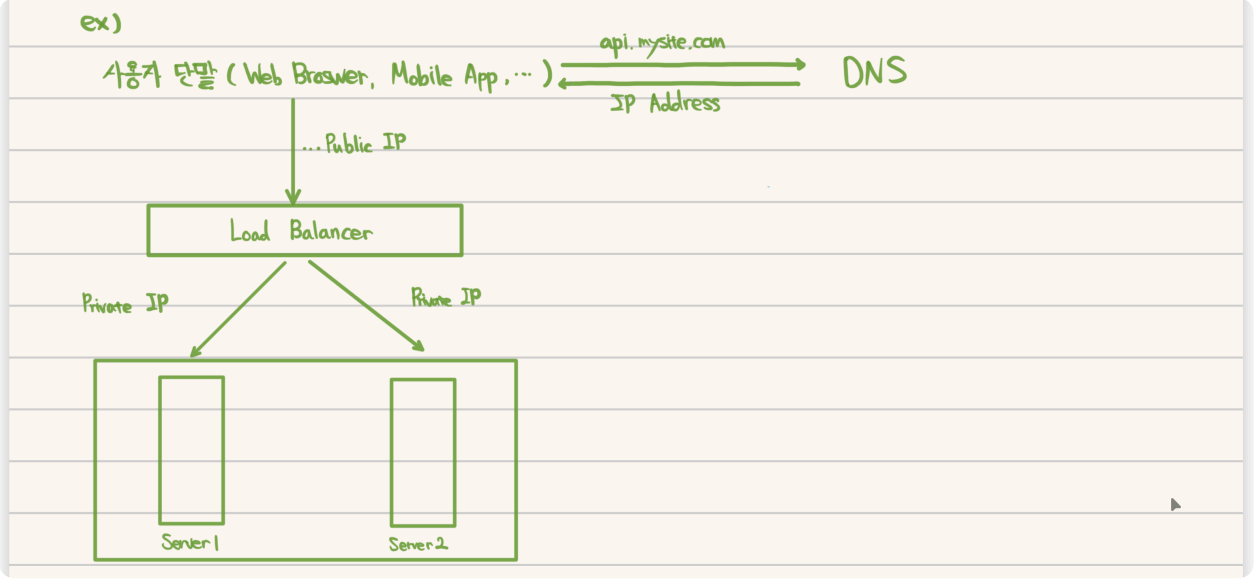
서버에 유입되는 **트래픽의 양이 적을 때**는 **수직적 확장**이 좋은 선택이며 단순하지만 몇 가지 큰 단점이 있다.

- 한 대의 서버에 CPU 나 메모리를 무한대로 증설할 방법이 없다
- 장애에 대한 자동복구 방안이나 다중화 방안을 제시하지 않는다
  : 서버에 장애가 발생하면 웹 사이트나 앱이 완전히 중단된다.

이러한 단점 때문에 대규모 애플리케이션을 지원하는데는 수평적 규모 확장법이 더 적절할 수 있다. 또한 너무 많은 사용자가 접속하여 웹 서버가 한계 상황에 도달하게 되면 응답 속도가 느려지거나 서버 접속이 불가능해질 수도 있다. 이런 문제를 해결하는데는 **부하 분산기 혹은 로드 밸런서**를 도입해보자.

1.3.1 로드 밸런서 : Load Balancer

로드 밸런서는 부하 분산 집합에 속한 웹 서버들에게 트래픽 부하를 고르게 분산하는 역할을 한다.



사용자는 LoadBalancer의 공개 IP 주소 (Public IP Address) 로 접속한다.
보안을 위해 서버 간 통신에는 사설 IP 주소 (private IP address) 가 이

용된다.
사설 IP 주소 같은 네트워크에 속한 서버 사이의 통신에만 쓰일 수 있는 IP 주소로 인터넷을 통해서는 접속할 수 없다.

예제의 그림처럼 웹 서버를 추가하면 웹 계층의 **가용성**은 향상된다.

> **가용성** : 웹 서버 일부에 장애가 발생해도 사용자가 계속 서비스를 이용할 수 있는 정도

- 서버1 이 다운되면 모든 트래픽은 서버2로 전송된다
  : 웹 사이트 전체가 다운되는 일이 방지된다
  : 부하를 나누기 위해 새로운 서버를 추가할 수 있다
- 두 대의 서버로 트래픽을 감당할 수 없는 시점이 오는데 로드밸런서가 있어 보다 나은 대처를 할 수 있다.
  : 더 많은 웹 서버를 추가하면 자동적으로 트래픽을 분산하기 시작한다.

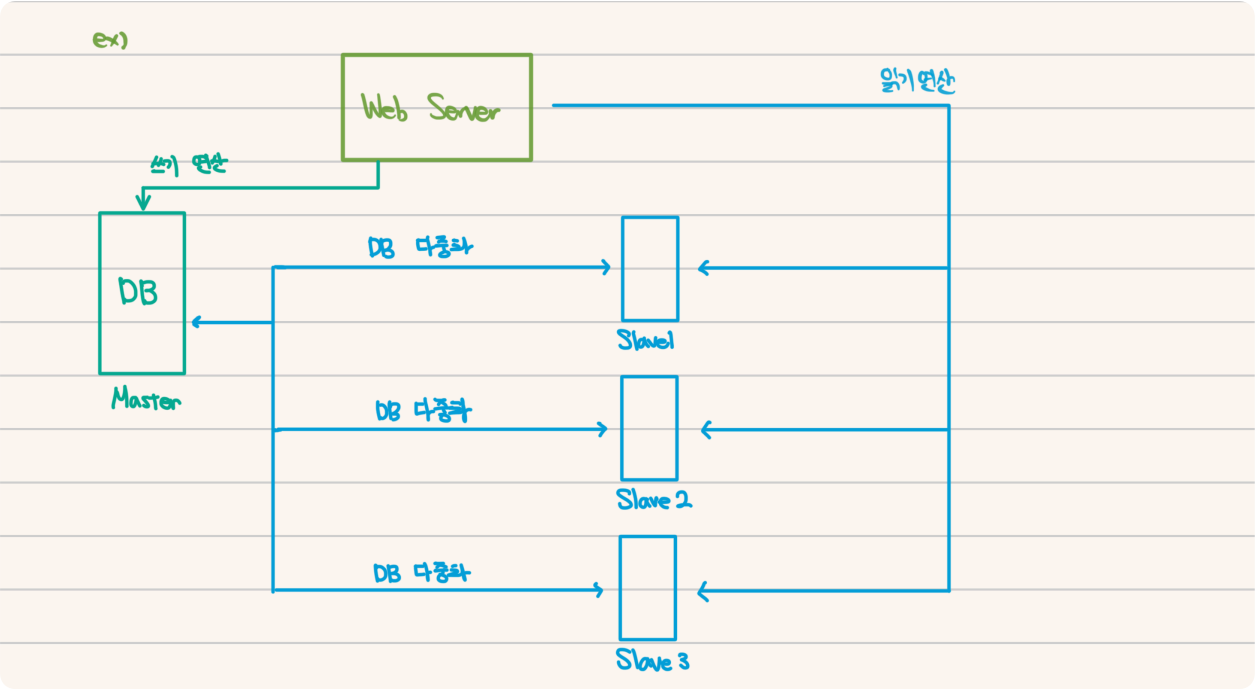
1.3.2 데이터베이스 다중화

많은 DBMS 가 다중화를 지원한다. 보통 서버 사이에 **Master - Slave** 관계를 설정한다. 여기서 마스터는 데이터 원본이고 슬래이브는 사본이다.

- **Master** : 쓰기 연산만 지원
- **Slave** : Master 로부터 그 사본을 전달하며 읽기 연산만을 지원한다.

```
insert, delete, update : Master
: 대부분의 애플리케이션은 읽기 연산의 비중이 쓰기 연산보다 훨씬 높다
```

보통 Master 보다 Slave 의 수가 더 많다

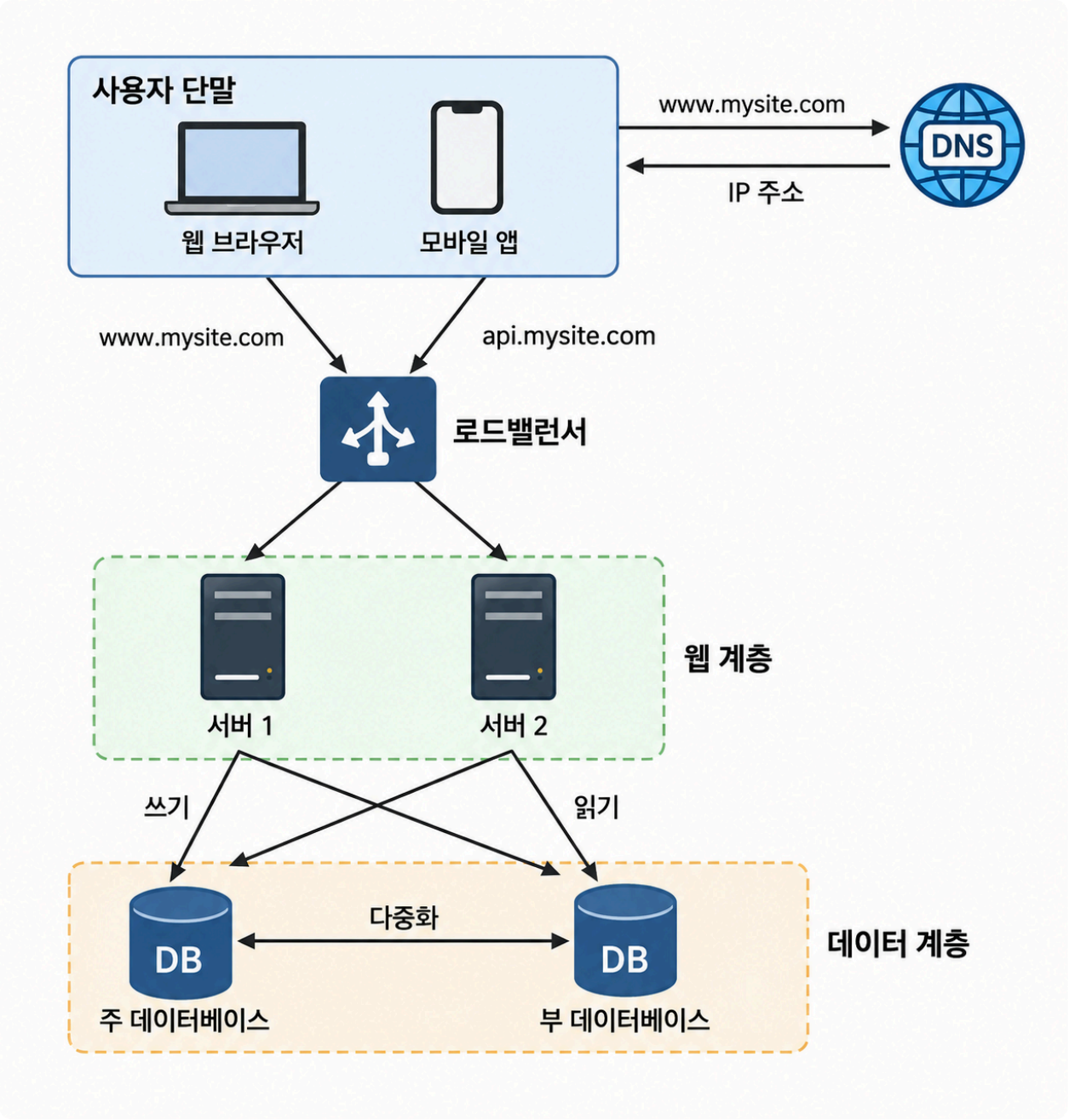


- 변경연산을 마스터로만 전달하고 읽기 연산을 슬래이브로 전달하면 병렬로 질의할 수 있는 쿼리 수가 늘어나 성능이 좋아진다.
- DB 서버 일부가 파괴되어도 데이터가 보존될 수 있다
- 하나의 서버에 장애가 발생해도 다른 서버에 있는 데이터 가져와 계속 서비스가 가능하다

1.3.2.1 만약 DB 서버 가운데 하나가 다운된다면?

- 부 서버가 한 대뿐이라면?
  : 읽기 연산은 한시적으로 모두 마스터로 전달되며 새로운 슬래이브가 장애서버를 대체할 것 이다.
- 주 DB 서버가 다운된다면?
  : 한 대의 슬래이브만 있는 경우 그 슬래이브가 새로운 마스터가 되며 새로운 슬래이브가 추가될 것이다.

하지만 **슬래이브에 보관된 데이터가 최신 상태가 아닐 수 있다.**
없는 데이터는 복구 스크립트를 돌려서 추가해야하며 다중 마스터나 원형 다중화 방식을 도입할 수 있지만 구성은 훨씬 복잡하다.



- 사용자는 DNS 로부터 로드 밸런서의 공개 IP 주소를 받는다
- 사용자는 해당 IP 주소를 사용해 로드 밸런서에 접속한다
- HTTP 요청은 서버1이나 서버2로 전달한다
- 웹 서버는 사용자의 데이터를 부 데이터베이스 서버에서 읽는다
- 웹 서버는 데이터 변경 연산을 주 DB 로 전달하고 추가·삭제·갱신 등이 이에 해당한다

1.4 캐시

캐시는 값 비싼 연산 결과 또는 자주 참조되는 데이터를 메모리 안에 두고 뒤이은 요청이 보다 빨리 처리될 수 있도록 하는 저장소이다. **애플리케이션의 성능은 데이터베이스를 얼마나 자주 호출하느냐에 크게 좌우되는데, 캐시는 그런 문제를 완화할 수 있다.**

1.4.1 캐시 계층

캐시 계층은 데이터가 잠시 보관되는 곳으로 데이터베이스보다 훨씬 바르다.
이러한 계층을 활용하면 다음과 같은 이점이 있다.

- 성능이 개선된다
- 데이터베이스의 부하를 줄일 수 있다
- 캐시 계층의 규모를 독립적으로 확장시키는 것도 가능해진다

요청을 받은 웹 서버는 캐시에 응답이 저장되어있는지를 본다.
만일 저장되어있다면 해당 데이터를 클라이언트에 반환한다. 없는 겨웅에는 데이터베이스 질의를 통해 데이터를 찾아 캐시에 저장한 뒤 클라이언트에 반환한다.

이 때 캐시할 데이터의 종류, 크기, 엑세스 패턴에 맞는 캐시 전략을 선택하면 된다.

1.4.2 캐시 사용 시 유의할 점

> 캐시는 어떤 상황에 바람직할까?
> 데이터 갱신은 자주 일어나지 않지만 참조는 빈번하게 일어난다면 고려할 만 하다. 즉, 데이터의 변동성이 낮고 조회는 많이 되면 고려해볼만하다.

캐시는 데이터를 휘발성 메모리에 두므로 영속적으로 보관할 데이터르르 캐시에 두는 것은 바람직하지 않다. 만약 캐시 서버가 재시작된다면 캐시

에 적재한 데이터는 모두 사라진다.

캐시에 보관된 데이터의 만료 정책은 어떻게 되는가?
데이터의 저장소의 원본과 캐시 내의 사본이 같은지에 대한 여부다.
저장소의 원본을 갱신하는 연산과 캐시를 갱신하는 연산이 단일 트랜잭션으로 처리되지 않는 경우 이 일관성은 깨질 수 있다.

장애에는 어떻게 대처할 것인가?
캐시 서버를 한 대만 두는 경우 해당 서버는 **단일 장애지점(Single Point of Failure, SPOF)** 이 되어버릴 가능성이 있다.
즉, 어떤 특정 지점에서의 장애가 전체 시스템의 동작을 중단 시켜버릴 수 있는 경우 해당 지점을 단일 장애 지점이라고 부르며 SPOF 를 피하려면 여러 지역에 걸쳐 캐시 서버를 분산시켜야한다.

캐시 메모리는 얼마나 크게 잡을 것인가?
캐시 메모리가 너무 작으면 데이터가 너무 자주 캐시에서 밀려나버려 캐시의 성능이 떨어지게 된다. 이 때 캐시 메모리를 과하게 할당하는 것이다.이렇게 하면 캐시에 보관될 데이터가 갑자기 늘어났을 때 생길 문제도 방지할 수 있게 된다.

데이터 방출 정책은 무엇인가?
캐시가 꽉 차버리면 추가로 캐시에 데이터를 넣어야할 경우 기존 데이터를 내보내야한다. 이를 캐시 데이터 방출 정책이라고 하는데 가장 널리 쓰이는 것은 LRU, LFU 나 FIFO 등이 있다. 경우에 맞게 적용하면 된다.

1.5 콘텐츠 전송 네트워크 : CDN

> **CDN** 은 정적 콘텐츠를 전송하는데 쓰이는 지리적으로 분산된 서버의 네트워크이다. 이미지, 비디오 , CSS, Javascript 파일 등을 캐시 할 수 있다.

어떤 사용자가 웹 사이트를 방문하면 그 사용자에게 가장 가까운 CSN 서버가 정적 콘텐츠를 전달하게 된다. 당연하겠지만 사용자가 **CDN 서버로부터 멀면 멀수록 웹 사이트는 천천히 로드 될 것이다.**

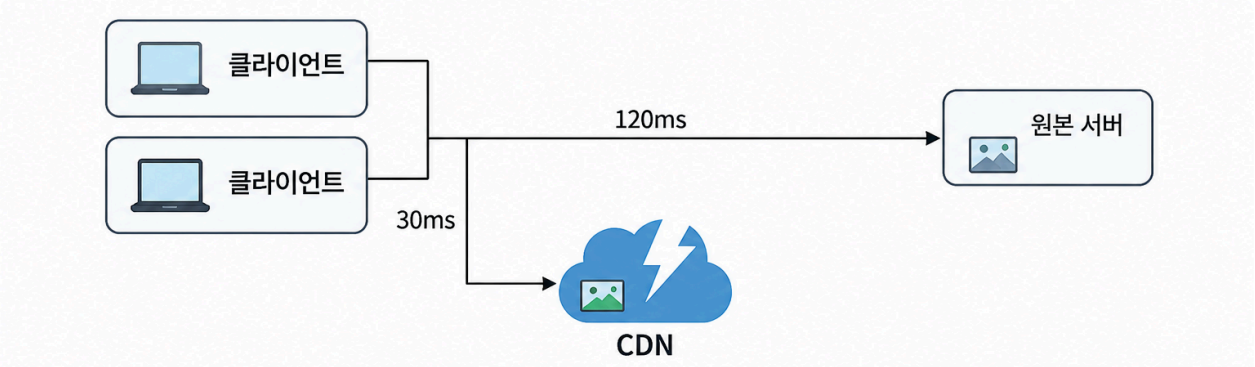


그림 1-9 CDN이 사이트 로딩 시간울 개선하는 예

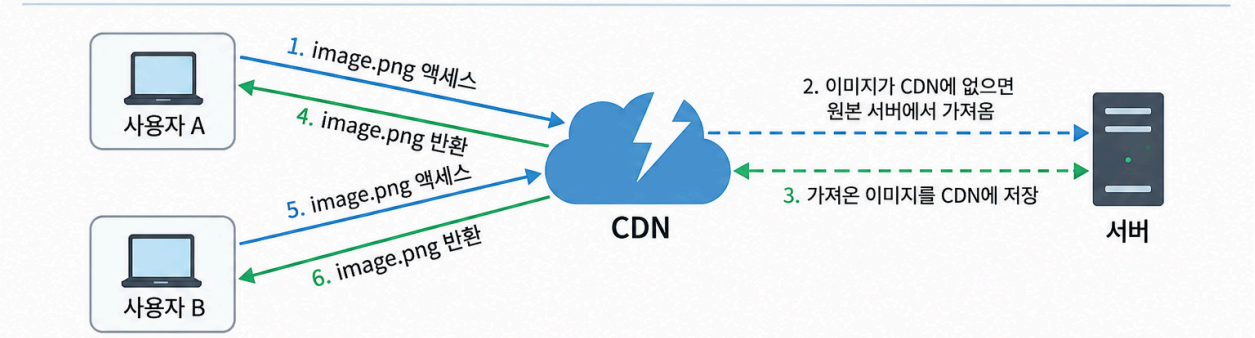


그림 1-10 CDN의 동작 과정

- 사용자 A가 이미지를 URL을 이용해 image.png 에 접근한다.
- CDN 서버의 캐시에 해당 이미지가 없는 경우 서버는 원본 서버에 요청하여 파일을 가져온다.
- 원본 서버가 파일을 CDN에 반환한다
- CDN 서버는 파일을 캐시하고 사용자 A에게 반환한다.

1.5.1 CDN 사용 시 고려해야할 사항

비용

CDN은 보통 제 3 사업자에 의해 운영되며 CDN으로 들어가고 나가는 데이터 전송 양에 따라 요금을 내게 된다.

적절한 만료 시한 설정

콘텐츠의 만료시점이 너무 길면 콘텐츠의 신선도는 떨어질 것이고 너무 짧으면 원본 서버에 빈번히 접속하게 되어서 좋지 않다.

CDN 장애에 대한 대처 방한

만약 CDN 자체가 죽었을 경우 웹사이트 및 애플리케이션이 어떻게 동작해야하는지 고려해야한다. 이러한 문제를 감지하여 원본 서버로부터 직접 콘텐츠를 가져오도록 하는 클라이언트를 구성하는 것이 필요할 수 있다.

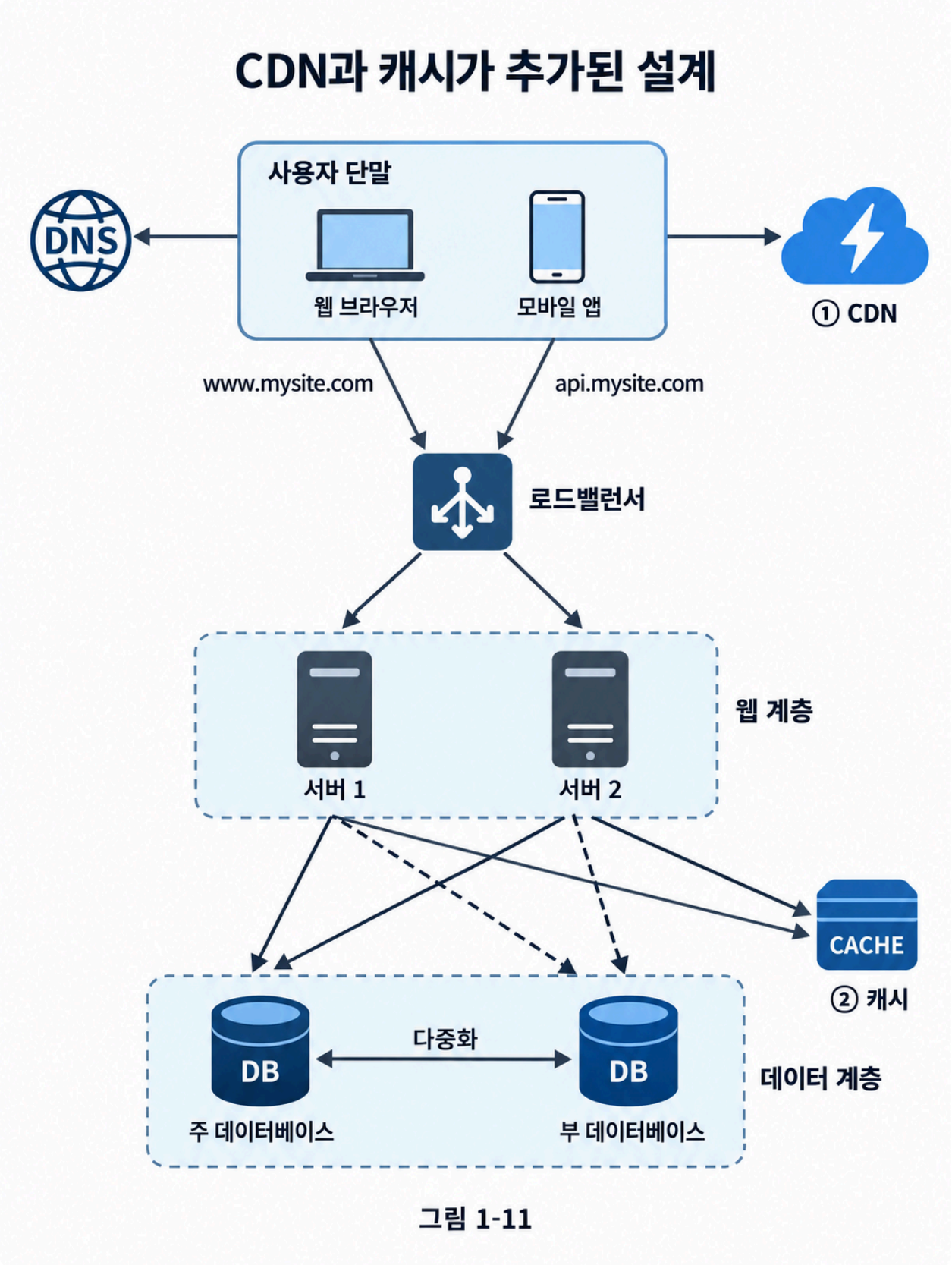


그림 1-11

1.6. Stateless 웹 계층

웹 계층을 수평적으로 확장하는 방법 즉, 서버를 증설을 해볼 수 있다. 이를 위해서는 서버가 상태정보(사용자 세션 데이터 등)을 웹 계층에서 제거해야한다. 상태 정보를 관계형 데이터 베이스나 NoSQL 같은 지속성 저장소에 보관하고 필요할 때 가져오도록 하는 것이다. 이렇게 구성된 웹 계층을 **무 상태 웹 계층**이라고 부른다.

1.6.1 상태 정보 의존적인 아키텍처

상태 정보를 보관하는 서버는 클라이언트 정보, 즉 상태를 유지하여 요청들 사이에 공유되도록 한다. 반면 무상태 서버에는 이러한 장치가 없고 다음과 같은그림은 상태 정보 의존적인 아키텍처를 보여준다.

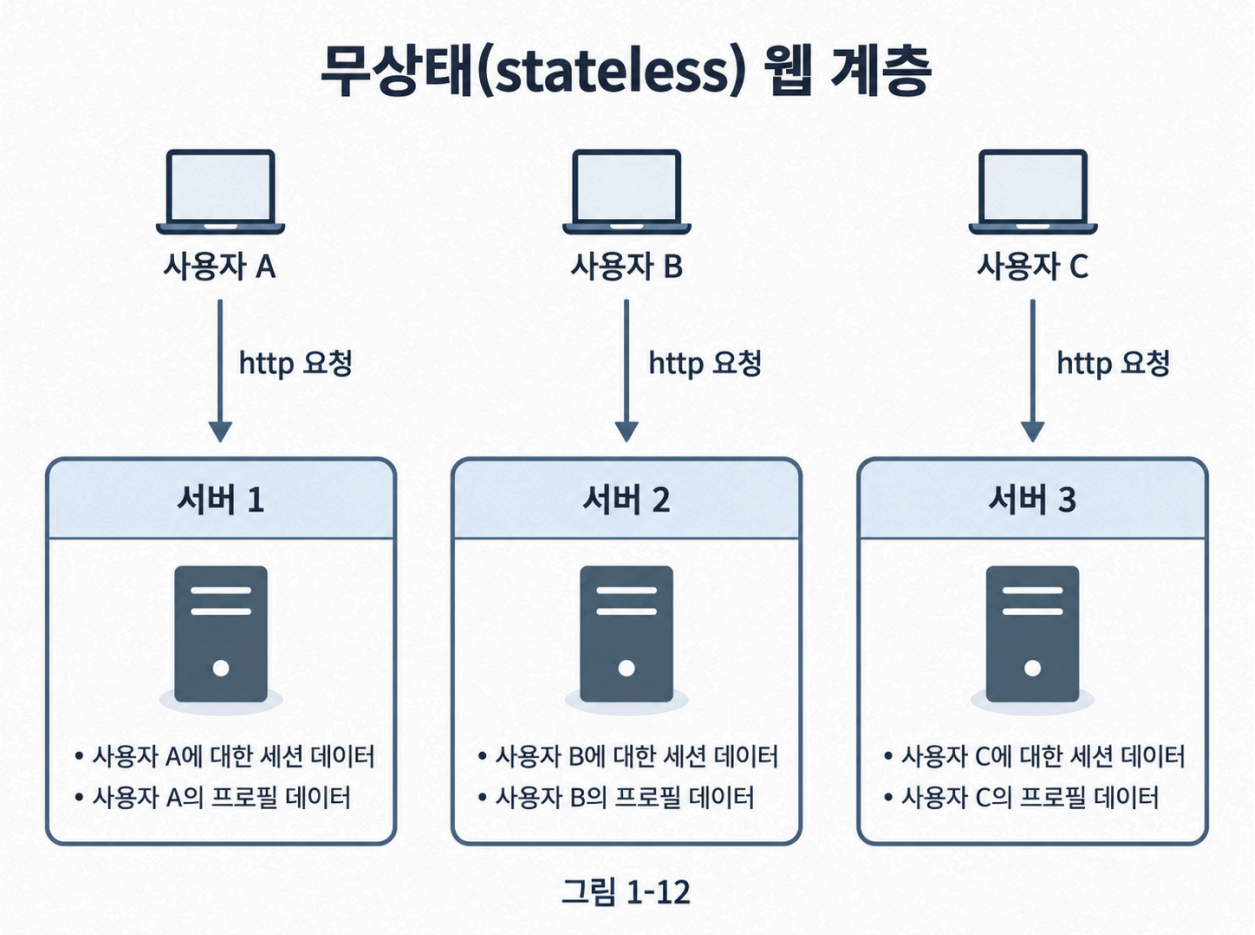


그림 1-12

상태를 갖고 있는 서버에는 다음과 같은 문제점이 존재한다.

- 사용자 A를 인증하기 위해 HTTP 요청은 반드시 서버1로 전송되어야 한다.
  : 다른 서버로 전송되면 인증에 실패한다.

: 사용자가들은 각각에 맞는 서버로 반드시 올바르게 전송되어야한다
: 즉, 클라이언트로부터의 요청은 항상 같은 서버로 전송되어야한다

- 로드밸런서의 고정 세션 기능
  : 이는 로드밸런서에 부담을 준다
  : 뒷단에 서버를 추가하거나 제거하기도 까다로워진다
  : 서버의 장애를 처리하기도 복잡하다

1.6.2 무상태 아키텍처

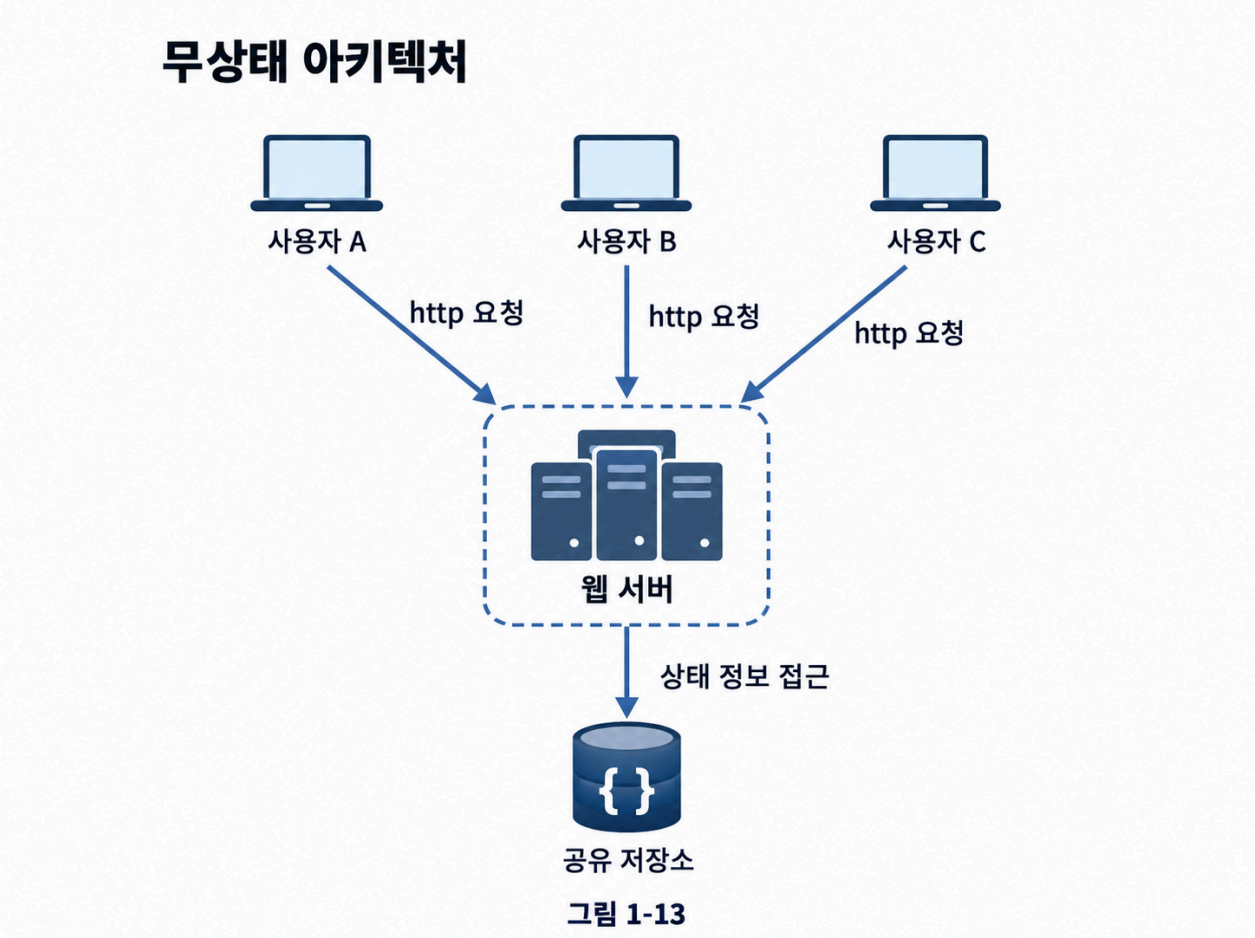


그림 1-13

이 구조는 사용자로부터의 HTTP 은 어떤 웹 서버로도 전달될 수 있다.
웹 서버는 상태 정보가 필요한 경우 공유 저장소로부터 데이터를 가져온다.
이런 구조는 단순하고 안정적이며 규모 확정이 쉽다.

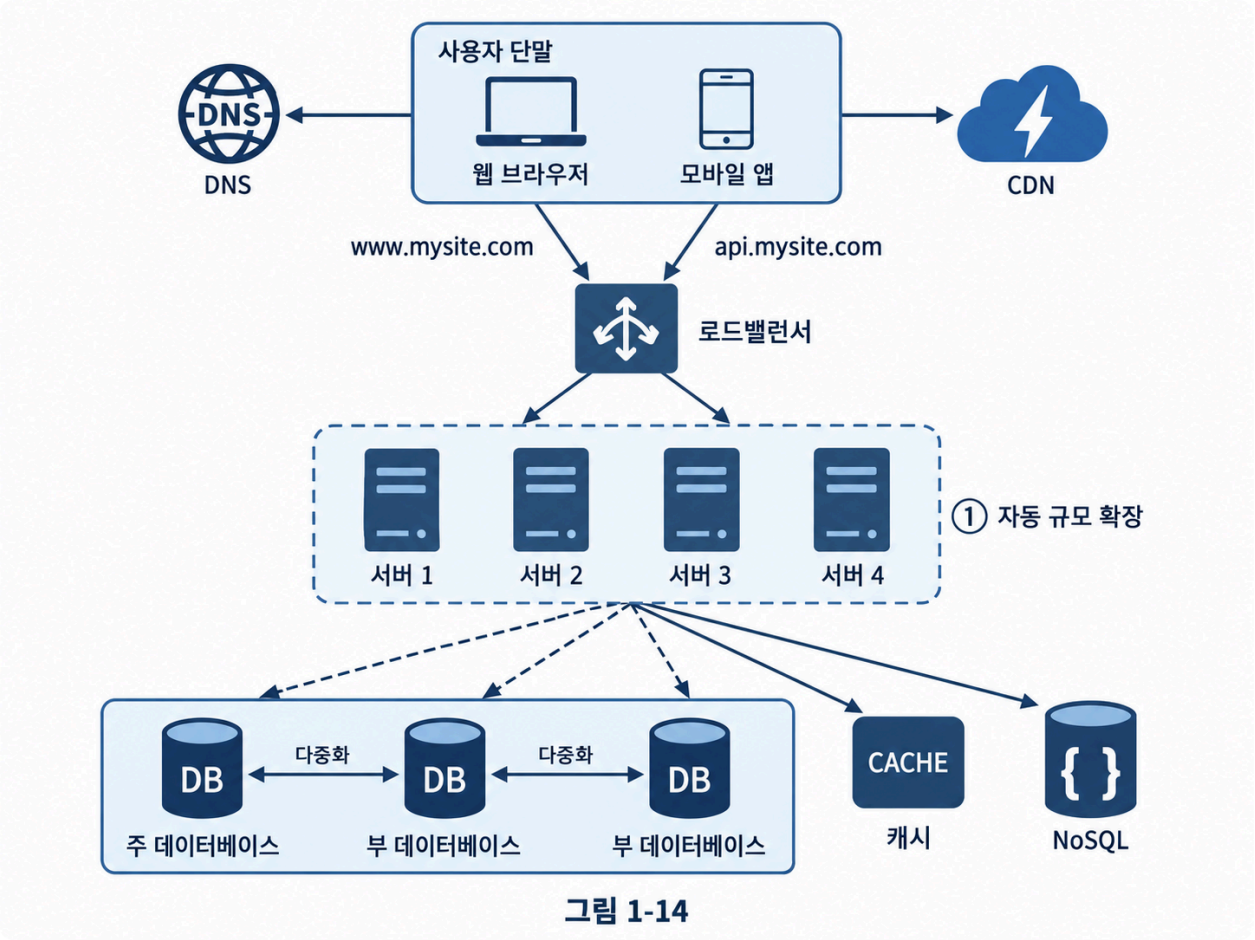


그림 1-14

이 **공유 저장소**는 관계형 데이터베이스일 수 있고 레디스 같은 캐시 시스템일 수도 있으며 NoSQL 일 수도 있다. 여기서는 NoSQL 을 사용하는데 규모 확장이 간편해서이다.

(1) 자동 규모 확장은 트래픽의 양에 따라 웹 서버를 자동으로 추가하거나 삭제하는 기능을 뜻한다. 상태 정보가 웹 서버들로부터 제거되었으므로 트래픽 양에 따라 웹 서버를 넣거나 빼기만하면 자동으로 규모 확장을 할 수 있게 되었다.

만약 만든 사이트가 매우 빨리 성장하여 전 세계 사용자의 이목을 받는 시점이 오게 되면 가용성을 높이고 전 세계 어디서도 쾌적하게 사용할 수 있도록 하기 위해서는 여러 데이터 센터를 지원하는 것이 필수다.

1.7 데이터 센터

다음의 그림은 두 개의 데이터 센터를 이용하는 사례다.
장애가 없는 상황에서 사용자는 가장 가까운 데이터 센터로 안내되는데 이

절차를 **지리적 라우팅(geoDNS-routing or geo-routing)** 이라고 부른다.
이는 사용자의 위치에 따라 도메인 이름을 어떤 IP 주소로 변환할지 결정할 수 있도록 해주는 DNS 서비스다.

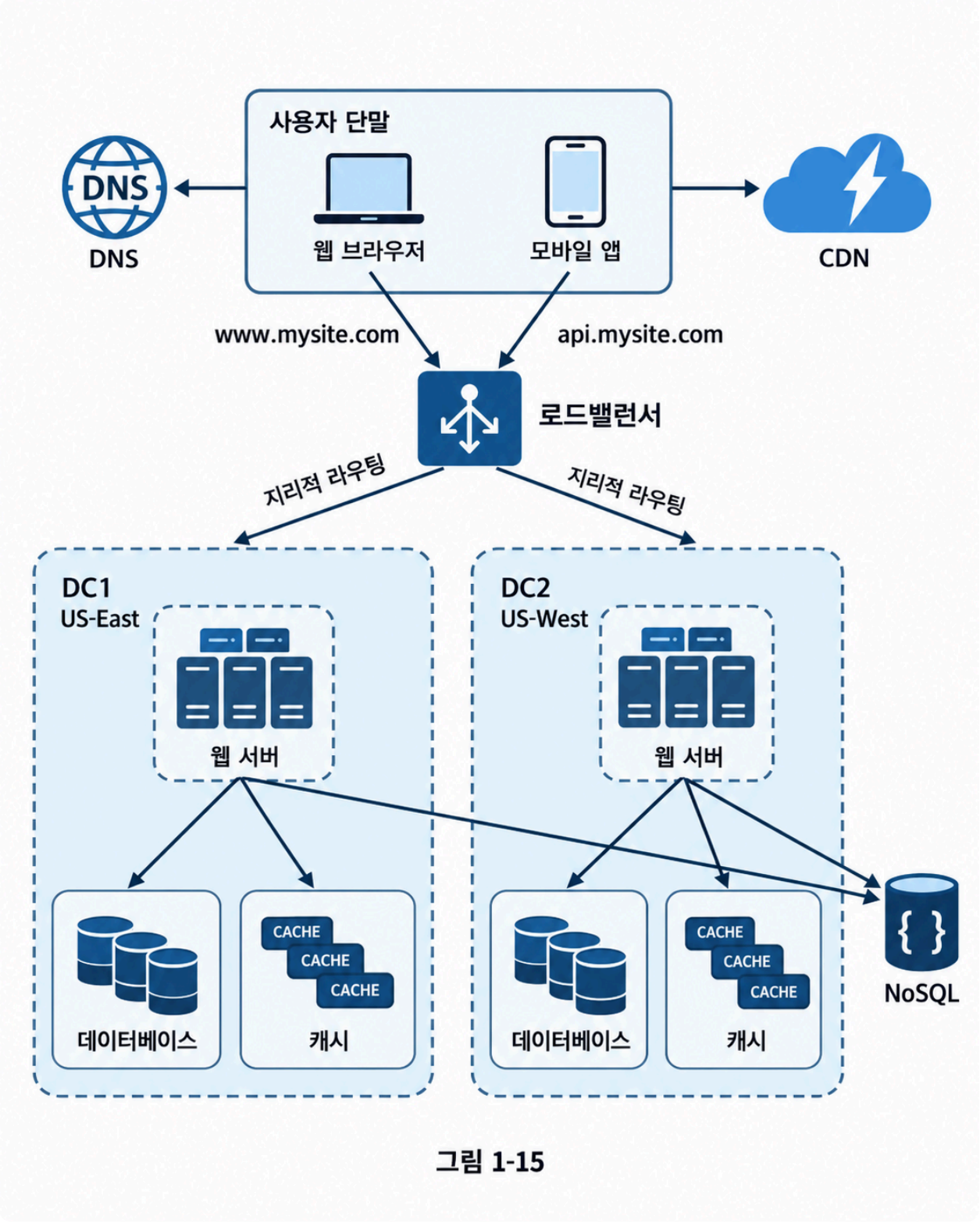


그림 1-15

데이터 센터 중 하나에 심각한 장애가 발생하면 모든 트래픽은 장애가 없는 데이터 센터로 전송된다.

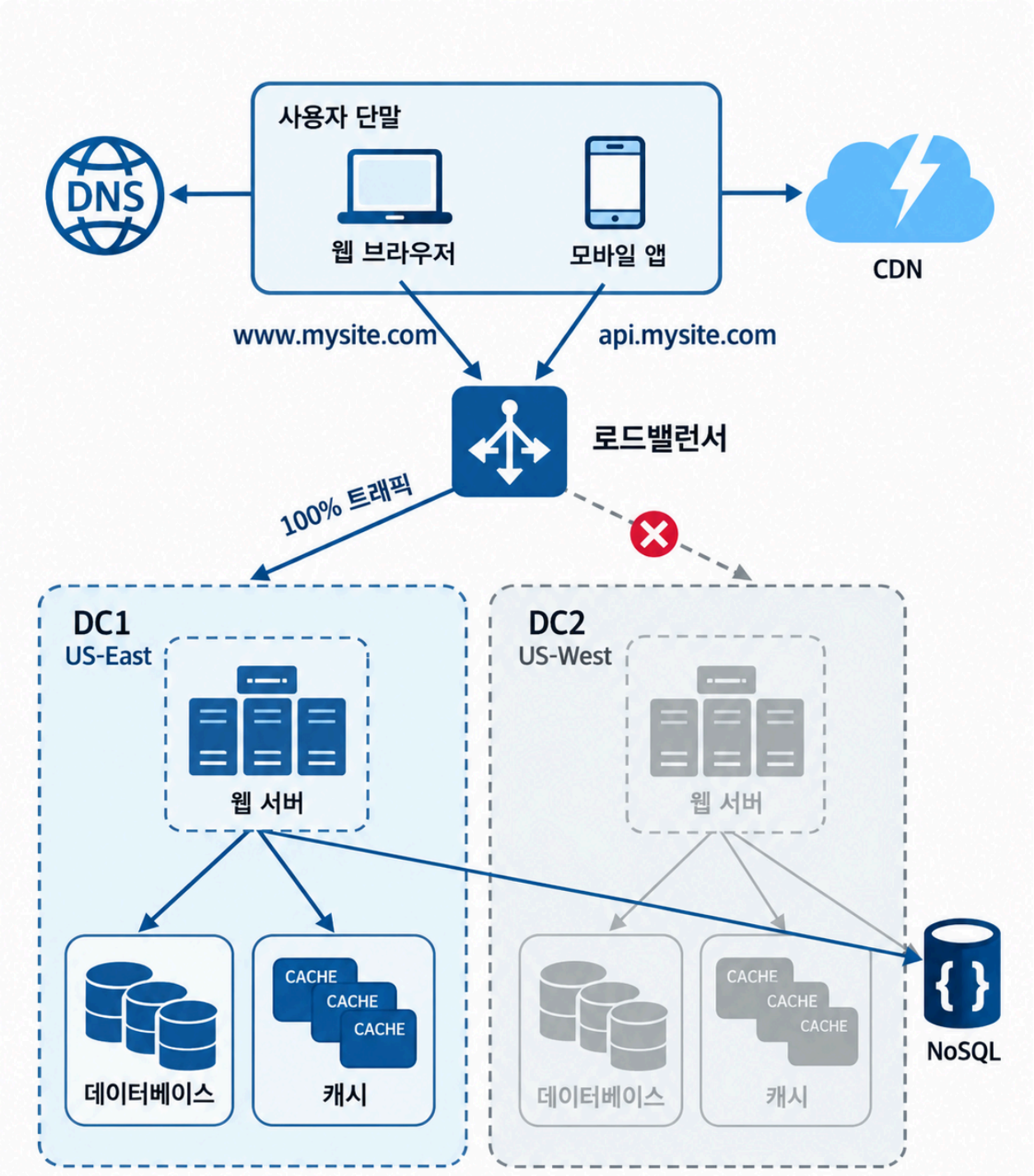


그림 1-16

다중 데이터 센터 아키텍처를 만드려면 몇 가지 기술적 난제를 해결해야한다.

- **트래픽 우회** : 올바른 데이터 센터로 트래픽을 보내는 효과적인 방법을 찾아야한다. GeoDNS 는 사용자에게서 가장 가까운 데이터 센터로 트래픽을 보낼 수 있도록 해준다.

- **데이터 동기화** : 데이터 센터마다 별도의 데이터베이스를 사용하고 있는 상황이라면 장애가 복구되어 트래픽이 다른 데이터베이스로 우회된다 해도 데이터 센터에는 찾는 데이터가 없을 수 있다. 보편적인 전략은 여러 데이터 센터에 걸쳐 다중화 하는 것이다.
- **테스트와 배포** : 여러 위치에서 테스트 해보는 것이 중요하다. 한편, 자동화된 배포 도구는 모든 데이터 센터에 동일한 서비스가 설치되도록 하는데 중요한 역할을 한다.

1.8 메시지 큐

메시지 큐는 메시지의 무손실을 보장하는 비동기 통신을 지원하는 컴포넌트이다.
메시지 큐의 기본 아키텍처는 간단하다. **생상자 또는 발행자(producer / publisher)** 이라고 불리는 입력 서비스가 메시지를 만들어 메시지 큐에 발행(publish) 한다. 큐에는 보통 소비자 혹은 구독자(consumer / subscriber) 라 불리는 서비스 혹은 서버가 연결되어있는데 메시지를 받아 그에 맞는 동작을 수행하는 역할을 한다.

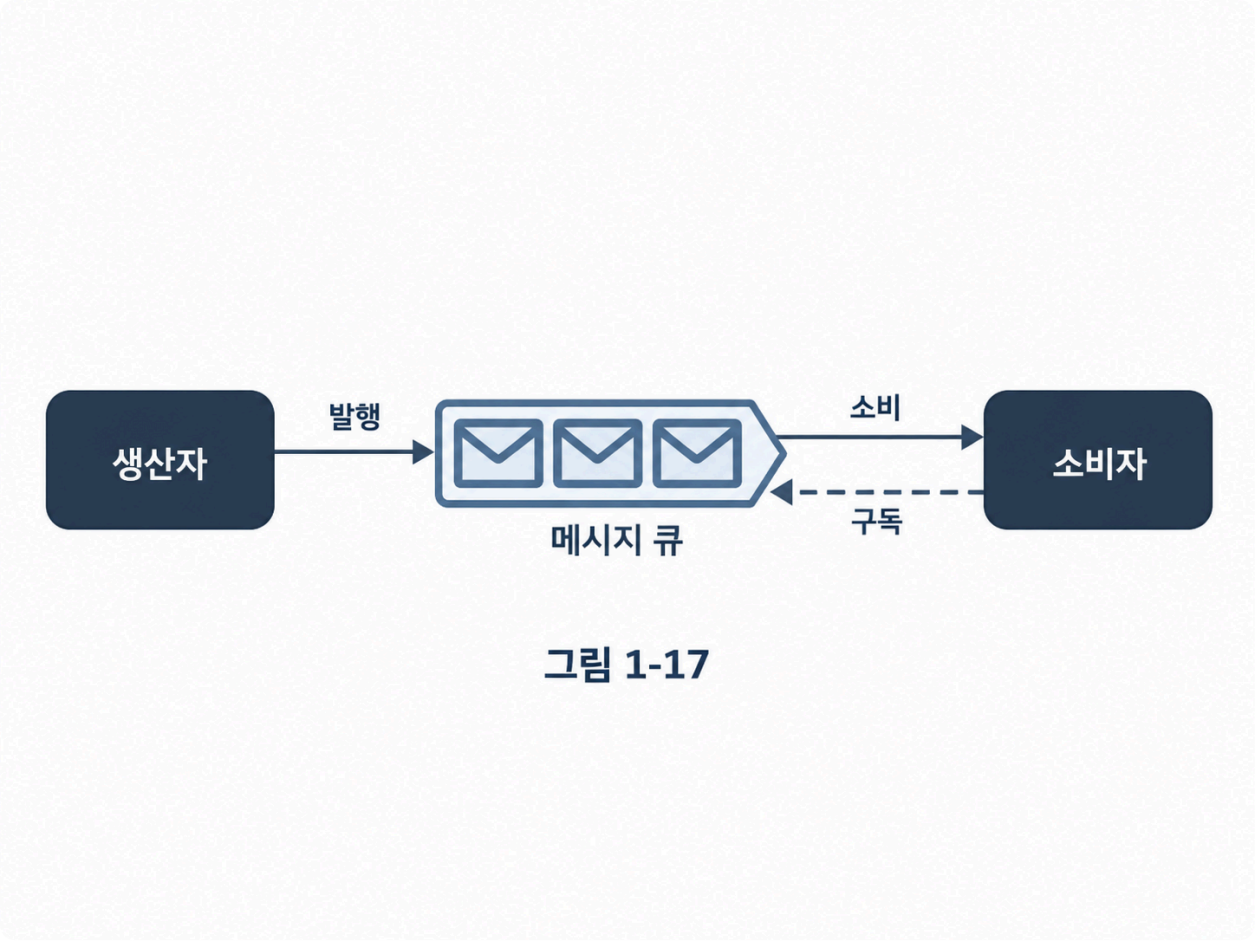


그림 1-17

메시지 큐를 이용하면 서비스 또는 서버 간 결합이 느슨해져 규모 확장성이 보장되어야하는 안정적 애플리케이션을 구성하기 좋다.
생상자는 소비자가 다운되어있어도 메시지를 발생할 수 있고 소비자는 생

상자 서비스가 가용한 상태가 아니더라도 메시지를 수신할 수 있다.

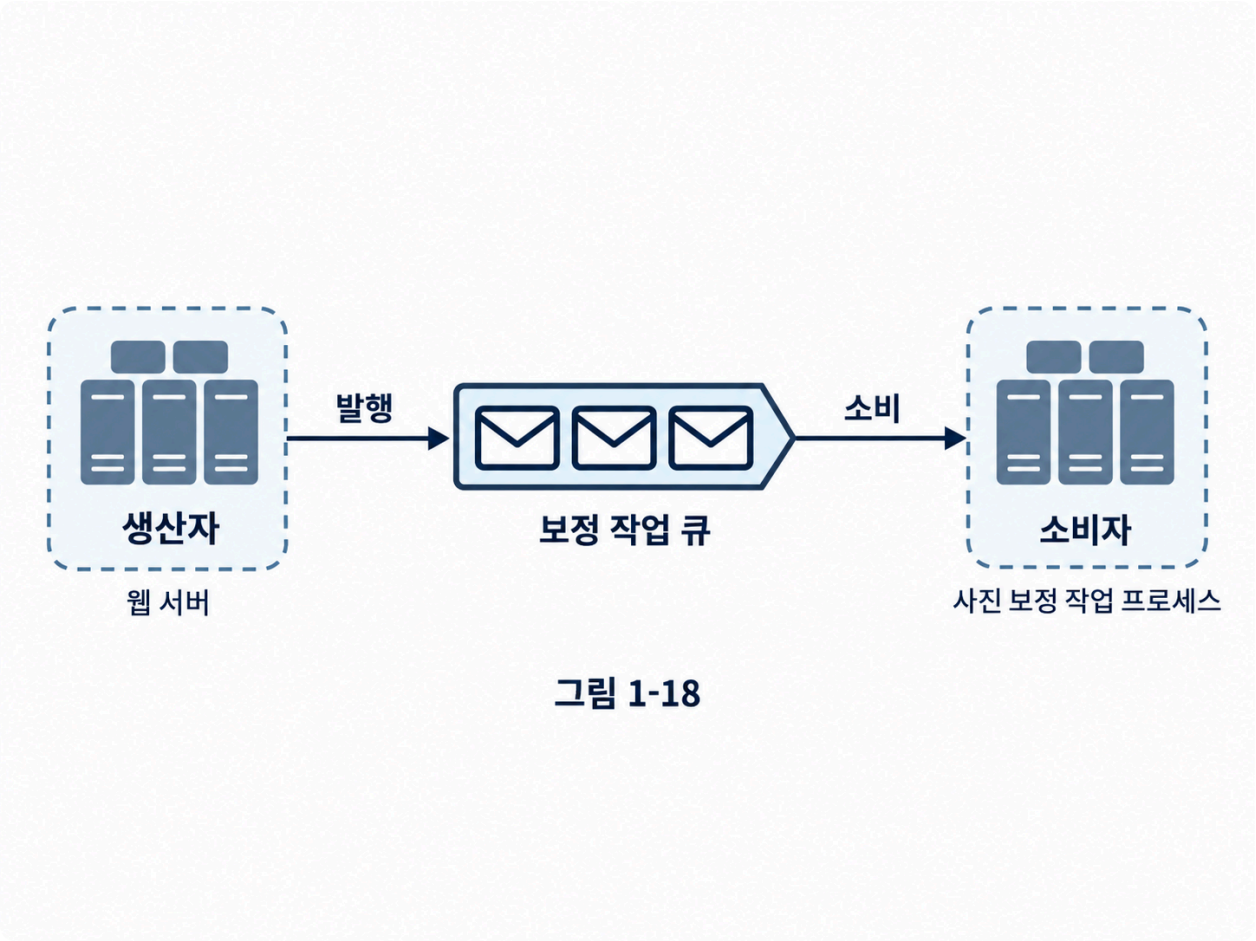


그림 1-18

1.9 로그, 메트릭 그리고 자동화

몇 개 서버에서 실행되는 소규모 웹 사이트를 만들 때 로그나 메트릭, 자동화 같은 것은 하면 좋지만 꼭 할 필요는 없다. 하지만 규모가 커지고 나면 필수적으로 투자해야한다.

- **로그** : 에러 로그를 모니터링 하는 것은 중요하다. 시스템의 오류와 문제들을 보다 쉽게 찾아낼 수 있도록 하기 때문이다. 에러 로그는 서버 단위로 모니터링 할 수도 있지만, 로그를 단일 서비스로 모아주는 도구를 활용하면 더 편리하다.
- **메트릭** : 메트릭을 잘 수집하면 사업 현황에 관한 유용한 정보를 얻을 수도 있고, 시스템의 현재 상태를 손쉽게 파악할 수 도 있다
    - **호스트 단위 메트릭**: CPU, 메모리, 디스크IO 에 관한 메트릭이 여기에 해당한다.

    - **종합 메트릭** : 데이터베이스 계층의 성능, 캐시 계층의 성능 같은 것이 여기 해당한다
    - **핵심 비즈니스 메트릭** : 일별 능동 상요자 수익, 재방문 같은 것이 여기에 해당한다.
- **자동화** : 시스템이 크고 복잡해지면 생상선을 높이기 위해 자동화 도구를 활용해야한다. 예를 들어 지속적 통합을 도와주는 도구를 활용할 수 있다.

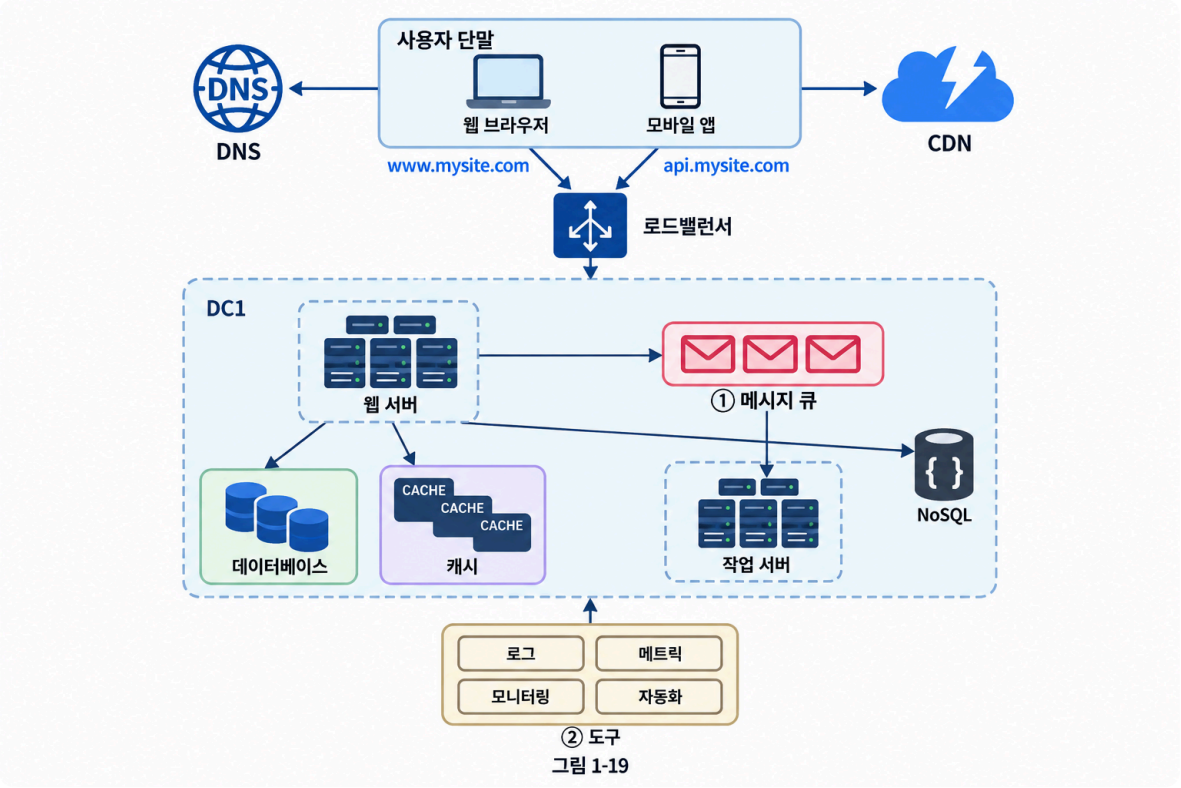


그림 1-19

1.10 데이터베이스의 규모 확장

데이터베이스의 규모를 확장하는데는 두 가지 접근법이 있다.
하나는 **수직적 규모 확장법** 이고 다른 하나는 **수평적 규모 확장법**이다.

1.10.1 수직적 확장 : Scale up

> 수직적 규모 확장은 Scale up 이라고도 불린다.

기존 서버에 더 많은 또는 고 성능의 자원(CPU, RAM, 디스크 등)을 증설하는 방법이다. 하지만 이러한 수직적 접근법에는 몇 가지 심각한 약점이 있다.

- 데이터베이스 서버 하드웨어에는 한계가 있으므로 CPU, RAM 등을 무한 증설할 수는 없다. 사용자가 계속 늘어나면 한 대 서버로는 결국 감당하기 어렵게 될 것이다.
- SPOF 로 인한 위험성이 크다
- 비용이 많이 든다. 고성능 서버로 갈수록 가격이 올라간다.

1.10.2 수평적 확장 : Sharding

데이터베이스의 수평적 확장은 샤딩(sharding) 이라고도 불리는데, 더 많은 서버를 추가함으로써 성능을 향상 시킬 수 있도록 한다.

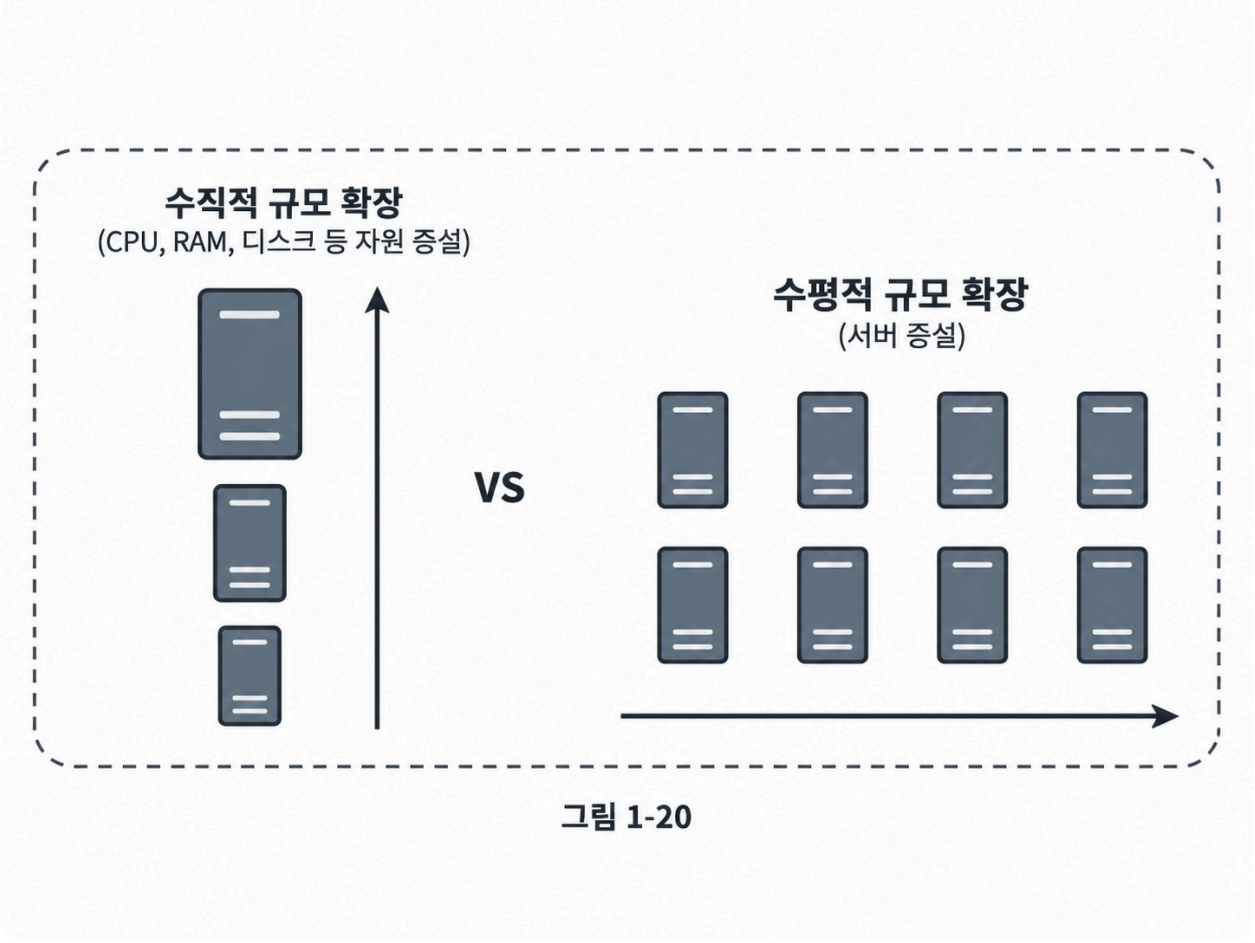


그림 1-20

샤딩은 대규모 데이터베이스를 `샤드(shard)` 라고 부르는 작은 단위로 분할되는 기술을 일컫는다. 모든 샤드는 같은 스키마를 쓰지만 샤드에 보관

되는 데이터 사이에는 중복이 없다.

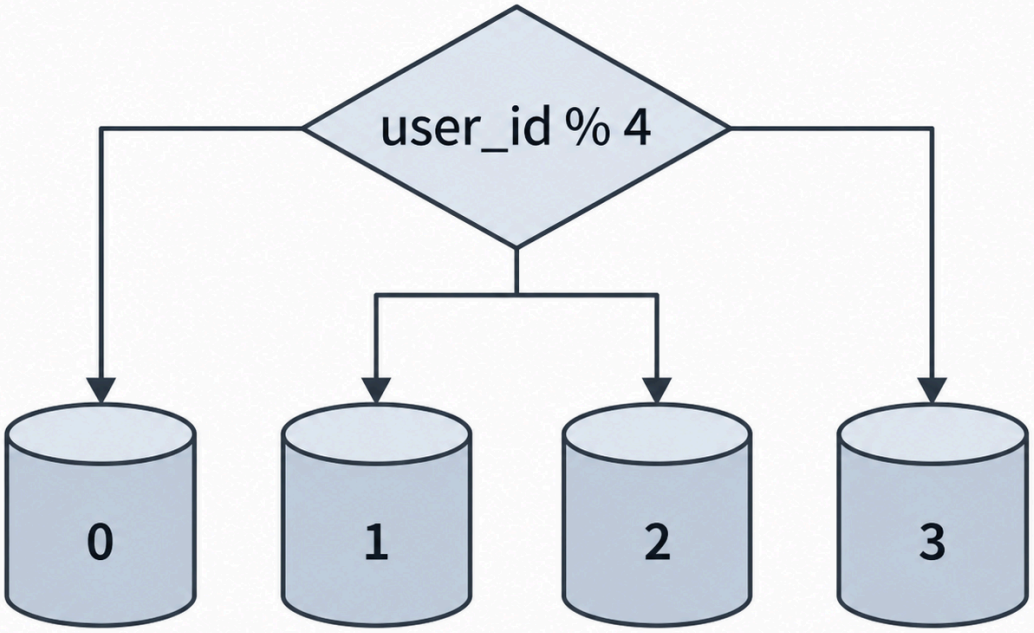


그림 1-21

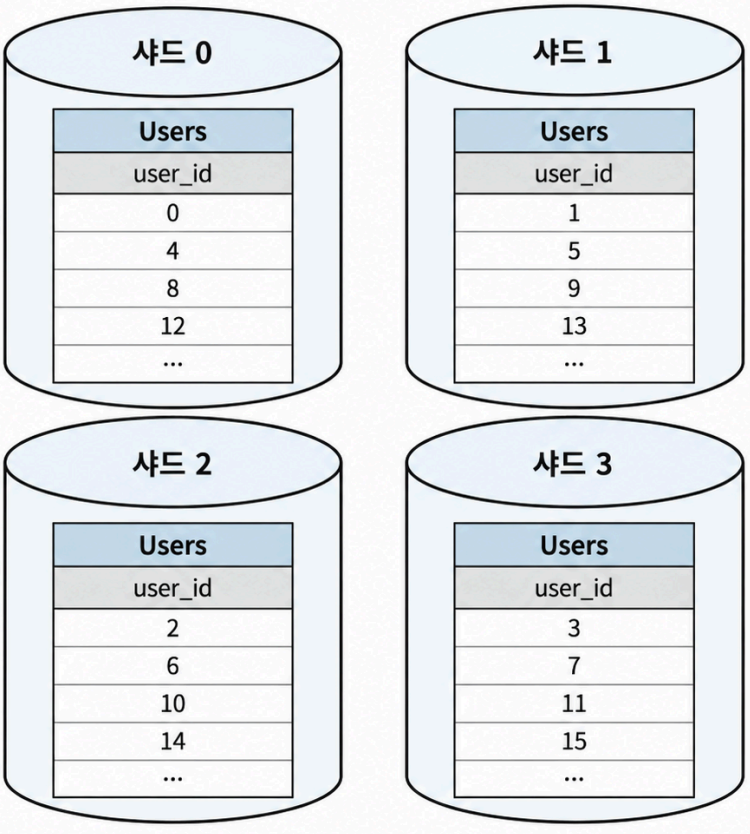


그림 1-22

샤딩 전략을 구현할 때 고려해야 할 가장 중요한 것은 **샤딩 키(Sharding Key)** 를 어떻게 정하느냐 하는 것이다. 샤딩 키는 파티션 키라고도 부르는데 데이터가 어떻게 분산될지 정하는 하나 이상의 컬럼으로 구성된다.

샤딩 키를 통해 올바른 데이터베이스에 질의를 보내어 데이터 조회나 변경을 처리하므로 효율을 높일 수 있따. 샤딩 키를 정할 때는 데이터를 고르게 분할 할 수 있도록 하는게 가장 중요하다. 하지만 샤딩을 도입하면 시스템이 복잡해지고 풀어야할 새로운 문제도 생긴다.

- **데이터의 재 샤딩**: 재 샤딩은 다음과 같은 경우에 필요하다. 첫 번째로 데이터가 너무 많아져서 하나의 샤드로는 더 이상 감당하기 어려울 때. 두 번째로 샤드 간 데이터 분포가 균등하지 못하여 어떤 샤드에 할당된 공간 소모가 다른 샤드에 비해 빨리 진행될 때. 샤드 소진이라고도 부르는 현상이 발생하면 샤드 키를 계산하는 함수를 변경하고 데이터를 재배치 해야한다.
- **유명인사 문제** : 핫스팟 키 문제라고도 부르는데 샤드에 질의가 집중되어 서버에 과부하가 걸리는 문제다. 예를 들어 리이디 가가, 저스틴 비버 등의 유명인사가 전부 같은 샤드에 저장되어있다면 특정 샤드는 읽기 연산 때문에 과부하가 걸리게 될 것이다. 이를 해결하려면 각각에 샤드 하나씩을 할당해야할 수도 있고 더 잘게 쪼개야할 수도 있다.
- **조인과 비정규화**: 일단 하나의 데이터베이스를 여러 샤드 서버로 쪼개고 나면 여러 샤드에 걸친 데이터를 조인하기가 힘들어진다. 이를 해결하는 방안은 데이터베이스를 비정규화하여 하나의 테이블에서 질의가 수행될 수 있도록 하는 것이다.

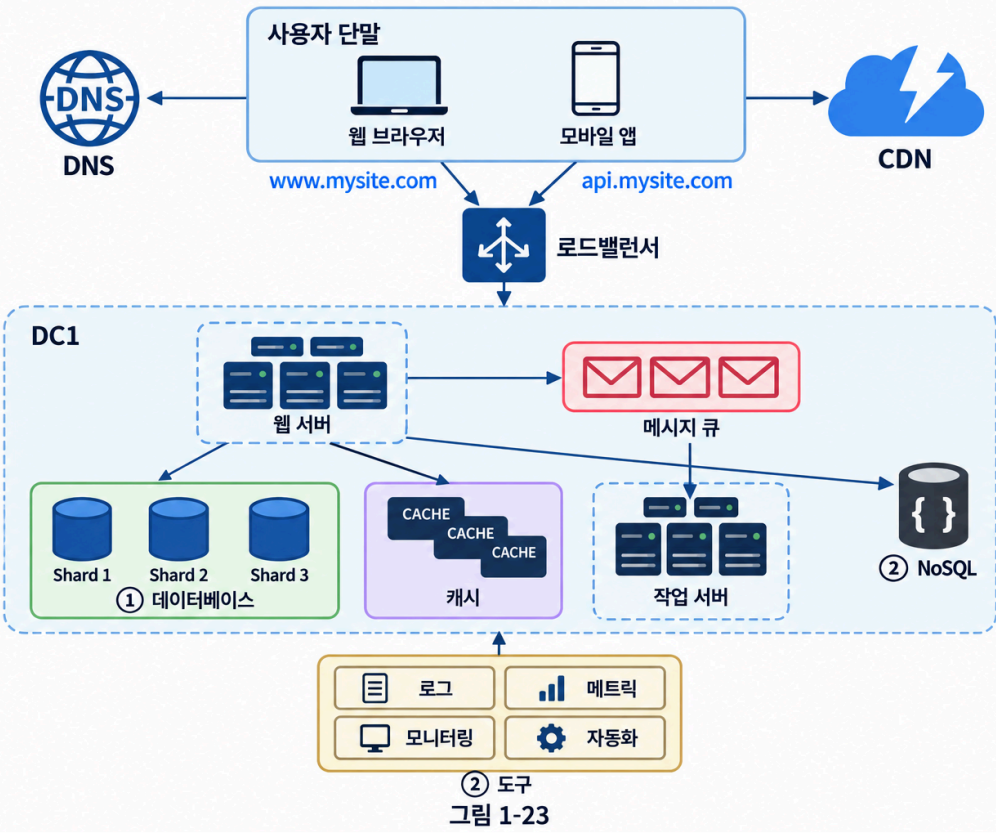

사용자 단말
웹 브라우저
모바일 앱
DNS
DNS
CDN
www.mysite.com
api.mysite.com
로드밸런서
DC1
웹 서버
메시지 큐
Shard 1
Shard 2
Shard 3
① 데이터베이스
CACHE
CACHE
CACHE
캐시
작업 서버
② NoSQL
로그
메트릭
모니터링
자동화
② 도구

그림 1-23